DATA 260 – Homework 6
MongoDB and Agent Memory Systems

Xuanhua Li
019122974

GitHub Repository:
https://github.com/carrovo/data260-2974

SID4: 2974
PORT_BASE: 8274
PREFIX: s2974
SEED: 2974
VERIFY_SEED: 262974
DOMAIN_ID: 6
Assigned Domain: Rental Housing Listings

Hardware: MacBook Air, Apple M4, 16 GB RAM
Local Model: qwen3:8b
Tagged Commit Hash: b317bad416ac5cf462c443ad6e5cdc98de9c59f6

Part 1 MongoDB Document API

Implementation Summary. I implemented a MongoDB-backed document API for rental housing listings using FastAPI, PyMongo, and Pydantic. The listings collection is stored in the s2974_doc database. Pydantic validates required fields, title length, enumerated values, and the availability date before a request reaches MongoDB.

Schema and database validation

Code snippet 1. Pydantic listing schema from code/hw6_schemas.py

```
class DocumentListingStatus(str, Enum):
    active = "active"
    pending = "pending"
    rented = "rented"


class DocumentPropertyType(str, Enum):
    apartment = "Apartment"
    house = "House"
    studio = "Studio"
    shared_room = "Shared Room"


class DocumentListingCreate(BaseModel):
    listingTitle: str = Field(min_length=1, max_length=100)
    description: str | None = Field(default=None, max_length=2000)
    status: DocumentListingStatus = DocumentListingStatus.active
    propertyType: DocumentPropertyType
    availableFrom: datetime
```

Code snippet 2. MongoDB collection validator from code/mongo_database.py

```
LISTING_VALIDATOR = {"$jsonSchema": {
    "required": ["listingTitle", "status", "propertyType",
                 "availableFrom", "createdAt", "updatedAt"],
    "properties": {
        "listingTitle": {"bsonType": "string", "maxLength": 100},
        "status": {"enum": ["active", "pending", "rented"]},
        "propertyType": {"enum": ["Apartment", "House", "Studio",
                                  "Shared Room"]},
        "availableFrom": {"bsonType": "date"}
    }
}}
```

CRUD endpoints

Method	Endpoint	Expected behavior
POST	/api/doc-listings	Create a listing and return HTTP 201
GET	/api/doc-listings	Return all listings with HTTP 200
GET	/api/doc-listings/{id}	Return one listing or HTTP 404
PUT	/api/doc-listings/{id}	Update supplied fields and updatedAt
DELETE	/api/doc-listings/{id}	Return HTTP 204 or HTTP 404

Code snippet 3. Representative create and update handlers

```
@router.post("", response_model=DocumentListingOut, status_code=201)
def create_document_listing(payload: DocumentListingCreate, db=Depends(...)):
    now = _utc_now()
    document = payload.model_dump(mode="python")
    document.update(createdAt=now, updatedAt=now)
    result = db.listings.insert_one(document)
    return _serialize(db.listings.find_one({"_id": result.inserted_id}))

@router.put("/{listing_id}", response_model=DocumentListingOut)
def update_document_listing(listing_id: str, payload: DocumentListingUpdate, ...):
    updates = payload.model_dump(exclude_unset=True)
    updates["updatedAt"] = _utc_now()
    result = db.listings.update_one({"_id": _object_id(listing_id)},
                                    {"$set": updates})
    if result.matched_count == 0:
        raise HTTPException(status_code=404,
                            detail="Document listing not found.")
```

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % STATUS=$(curl -sS \
  -o /tmp/hw6_create.json \
  -w "%{http_code}" \
  -X POST "http://127.0.0.1:8274/api/doc-listings" \
  -H "Content-Type: application/json" \
  -d '{
    "listingTitle": "Downtown San Jose Apartment",
    "description": "One-bedroom apartment near downtown San Jose with in-unit laundry.",
    "status": "active",
    "propertyType": "Apartment",
    "availableFrom": "2026-11-15T12:00:00Z"
  }')

echo "Student: Xuanhua Li | HW6 Part 1 - MongoDB CREATE"
echo "POST /api/doc-listings -> HTTP $STATUS"
python -m json.tool /tmp/hw6_create.json
Student: Xuanhua Li | HW6 Part 1 - MongoDB CREATE
POST /api/doc-listings -> HTTP 201
{
    "id": "6ad12d178725577b551b4939",
    "listingTitle": "Downtown San Jose Apartment",
    "description": "One-bedroom apartment near downtown San Jose with in-unit laundry.",
    "status": "active",
    "propertyType": "Apartment",
    "availableFrom": "2026-11-15T12:00:00Z",
    "createdAt": "2026-10-15T19:44:23.641000Z",
    "updatedAt": "2026-10-15T19:44:23.641000Z"
}
((.venv) ) carol@Carols-MacBook-Air data260-2974 % LISTING_ID=$(python -c 'import json; print(json.load(open("/tmp/hw6_create.json"))["id"])')
echo "LISTING_ID=$LISTING_ID"
LISTING_ID=6ad12d178725577b551b4939
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 1. POST created a MongoDB listing and returned HTTP 201.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 1 - MongoDB READ"
echo "GET all listings -> HTTP $LIST_STATUS"
python -c 'import json; d=json.load(open("/tmp/hw6_list.json")); print("Listing count:", len(d)); print("First listing ID:", d[0]["id"]); print("First
 listing title:", d[0]["listingTitle"])'
echo
echo "GET listing by ID -> HTTP $ONE_STATUS"
python -m json.tool /tmp/hw6_get_one.json
Student: Xuanhua Li | HW6 Part 1 - MongoDB READ
GET all listings -> HTTP 200
Listing count: 1
First listing ID: 6ad12d178725577b551b4939
First listing title: Downtown San Jose Apartment

GET listing by ID -> HTTP 200
{
    "id": "6ad12d178725577b551b4939",
    "listingTitle": "Downtown San Jose Apartment",
    "description": "One-bedroom apartment near downtown San Jose with in-unit laundry.",
    "status": "active",
    "propertyType": "Apartment",
    "availableFrom": "2026-11-15T12:00:00Z",
    "createdAt": "2026-10-15T19:44:23.641000Z",
    "updatedAt": "2026-10-15T19:44:23.641000Z"
}
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 2. GET returned the listing collection and a single document with HTTP 200.

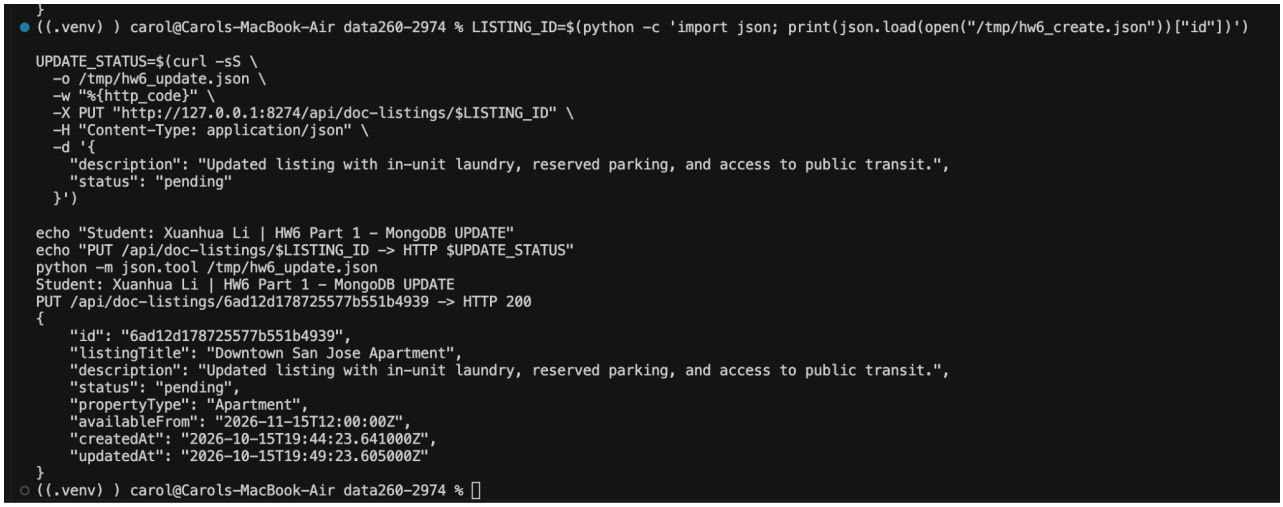


Figure 3. PUT updated the description and status while preserving the document ID.

Validation and deletion

Code snippet 4. Application-level validation and delete error handling

```
@field_validator("listingTitle")
@classmethod
def reject_blank_title(cls, value: str) -> str:
    cleaned = value.strip()
    if not cleaned:
        raise ValueError("listingTitle cannot be blank")
    return cleaned

if result.deleted_count == 0:
    raise HTTPException(status_code=404,
                        detail="Document listing not found.")
```

Invalid input	Validation result
Missing listingTitle	HTTP 422 Field required
propertyType = Condo	HTTP 422 Value must match the allowed enum
availableFrom = not-a-date	HTTP 422 Invalid datetime or date

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % MISSING_STATUS=$(curl -sS \
  -o /tmp/hw6_invalid_missing.json \
  -w "%{http_code}" \
  -X POST "http://127.0.0.1:8274/api/doc-listings" \
  -H "Content-Type: application/json" \
  -d '{
    "status": "active",
    "propertyType": "Apartment",
    "availableFrom": "2026-12-01T12:00:00Z"
  }')

ENUM_STATUS=$(curl -sS \
  -o /tmp/hw6_invalid_enum.json \
  -w "%{http_code}" \
  -X POST "http://127.0.0.1:8274/api/doc-listings" \
  -H "Content-Type: application/json" \
  -d '{
    "listingTitle": "Invalid Property Type",
    "status": "active",
    "propertyType": "Condo",
    "availableFrom": "2026-12-01T12:00:00Z"
  }')

DATE_STATUS=$(curl -sS \
  -o /tmp/hw6_invalid_date.json \
  -w "%{http_code}" \
  -X POST "http://127.0.0.1:8274/api/doc-listings" \
  -H "Content-Type: application/json" \
  -d '{
    "listingTitle": "Invalid Date Listing",
    "status": "active",
    "propertyType": "Apartment",
    "availableFrom": "not-a-date"
  }')
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 1 — Validation"
echo "Missing listingTitle -> HTTP $MISSING_STATUS"
python -c 'import json; e=json.load(open("/tmp/hw6_invalid_missing.json"))["detail"][0]; print("Field:", e["loc"][-1], "| Error:", e["msg"])'

echo "Invalid propertyType -> HTTP $ENUM_STATUS"
python -c 'import json; e=json.load(open("/tmp/hw6_invalid_enum.json"))["detail"][0]; print("Field:", e["loc"][-1], "| Error:", e["msg"])'

echo "Invalid availableFrom -> HTTP $DATE_STATUS"
python -c 'import json; e=json.load(open("/tmp/hw6_invalid_date.json"))["detail"][0]; print("Field:", e["loc"][-1], "| Error:", e["msg"])'
Student: Xuanhua Li | HW6 Part 1 — Validation
Missing listingTitle -> HTTP 422
Field: listingTitle | Error: Field required
Invalid propertyType -> HTTP 422
Field: propertyType | Error: Input should be 'Apartment', 'House', 'Studio' or 'Shared Room'
Invalid availableFrom -> HTTP 422
Field: availableFrom | Error: Input should be a valid datetime or date, invalid character in year
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 4. Missing, invalid-enum, and invalid-date inputs each returned HTTP 422.

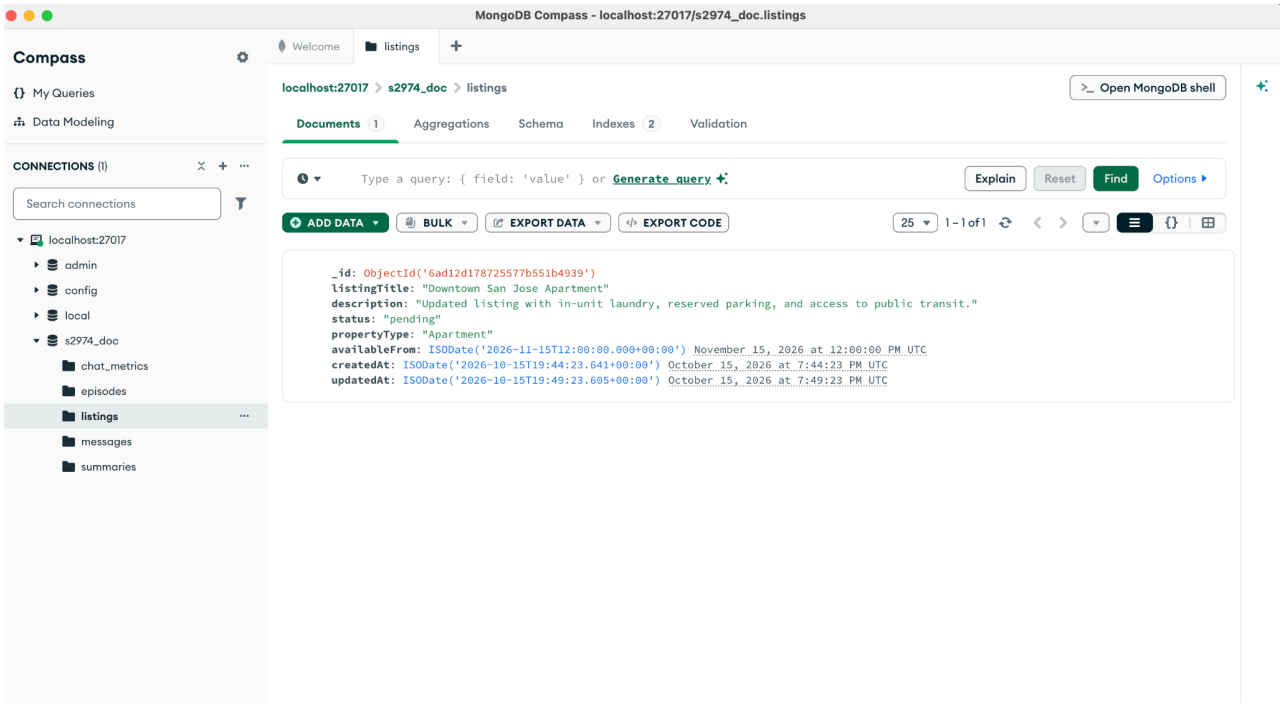


Figure 5. MongoDB Compass shows the updated document in s2974_doc.listings.

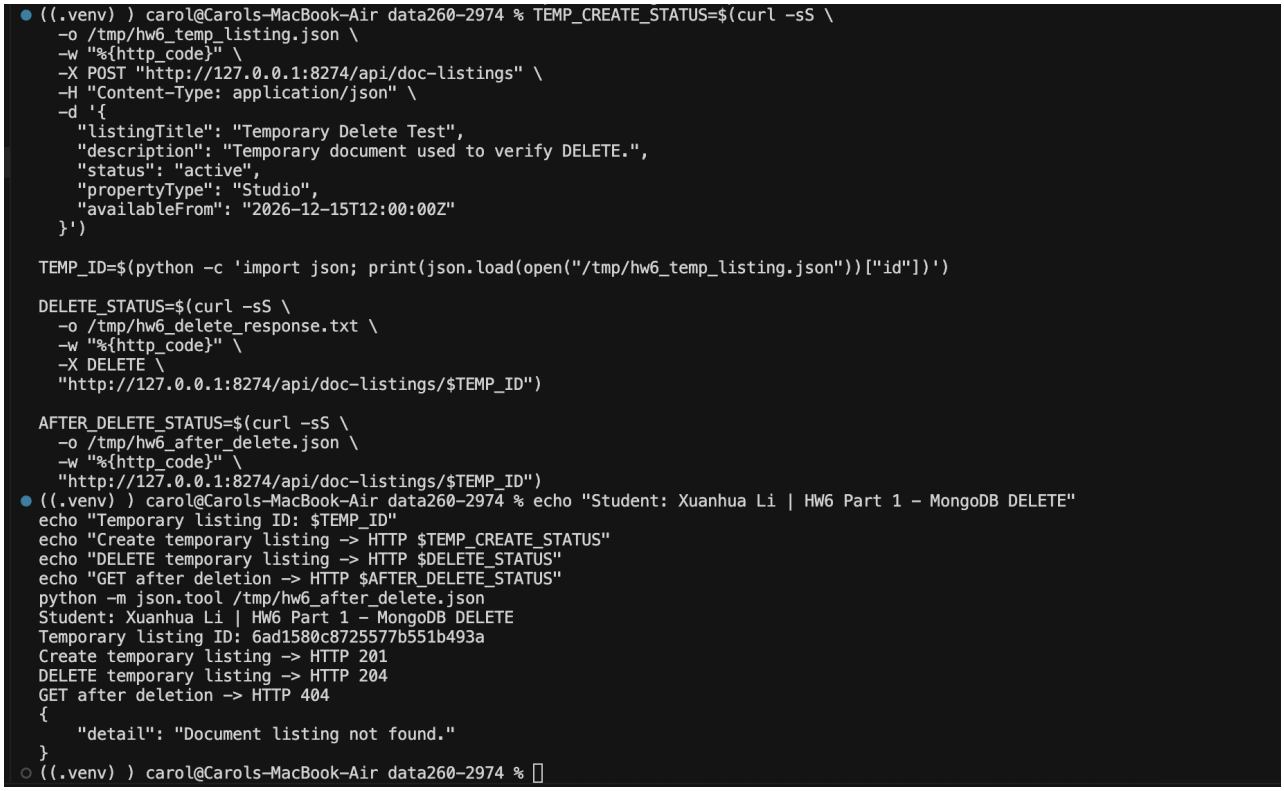


Figure 6. DELETE returned HTTP 204 and the follow-up GET returned HTTP 404.

Result. The evidence demonstrates all CRUD operations, database persistence, automatic timestamps, Pydantic validation, and not-found handling. CORS is enabled in code/main.py for the local front-end origins.

Part 2 Short Long and Episodic Memory

The MongoDB memory pipeline stores raw conversation turns, periodically generated summaries, and embedded episodic facts. For each chat request, it combines a short-term message window, the latest session and lifetime summaries, and the top-k relevant episodes before calling the local Ollama model.

MongoDB collections

Collection	Main fields	Purpose
messages	user_id, session_id, role, content, created_at	Store user and assistant turns
summaries	user_id, nullable session_id, scope, text, created_at	Store session and lifetime summaries
episodes	user_id, session_id, fact, importance, embedding, created_at	Store retrievable episodic facts

FastAPI endpoints

Endpoint	Purpose
POST /api/chat	Run the memory pipeline, call Ollama, save the reply, and return used-memory evidence
GET /api/memory/{user_id}	Return recent messages, summaries, and episodic facts
GET /api/aggregate/{user_id}	Return daily message counts and recent summaries

Code snippet 5. The three required memory endpoints

```
@router.post("/api/chat", response_model=ChatResponse)
def chat(payload: ChatRequest, db: Database = Depends(mongo_db_dependency)):
    return _service(db).chat(user_id=payload.user_id,
                             session_id=payload.session_id,
                             message=payload.message,
                             memory_mode=payload.memory_mode,
                             policy=payload.policy,
                             short_term_n=payload.short_term_n)

@router.get("/api/memory/{user_id}")
def get_memory(user_id: str, session_id: str | None = None, ...): ...

@router.get("/api/aggregate/{user_id}")
def get_aggregate(user_id: str, ...): ...
```

Memory logic

Code snippet 6. Short-term, long-term, episodic retrieval, and prompt assembly

```
prior_messages = repository.recent_messages(user_id, session_id,
                                            window_size + 1)[:-1]
session_summary, lifetime_summary = self._summary_text(user_id, session_id)
query_embedding = self.embedder.embed(message)
episodes = self.retrieve_episodes(user_id, query_embedding)

system_content = (
    f"{SYSTEM_PRIMER}\n\nLong-term summaries:\n{long_term_text}\n\n"
    f"Relevant episodic facts:\n{episodic_text}\n\n"
    f"Recent short-term conversation:\n{short_term_text}"
)
result = self.model.complete([
    {"role": "system", "content": system_content},
    {"role": "user", "content": message},
])
```

Code snippet 7. Episodic ranking and summarization trigger

```
for episode in self.repository.recent_episodes(user_id):
    similarity = cosine_similarity(query_embedding, episode["embedding"])
    episode["retrieval_score"] = (
        0.85 * similarity + 0.15 * episode["importance"]
    )

if user_message_count % settings.summarize_every_user_messages == 0:
    self._create_session_summary(user_id, session_id)
    self._maybe_create_lifetime_summary(user_id)
```

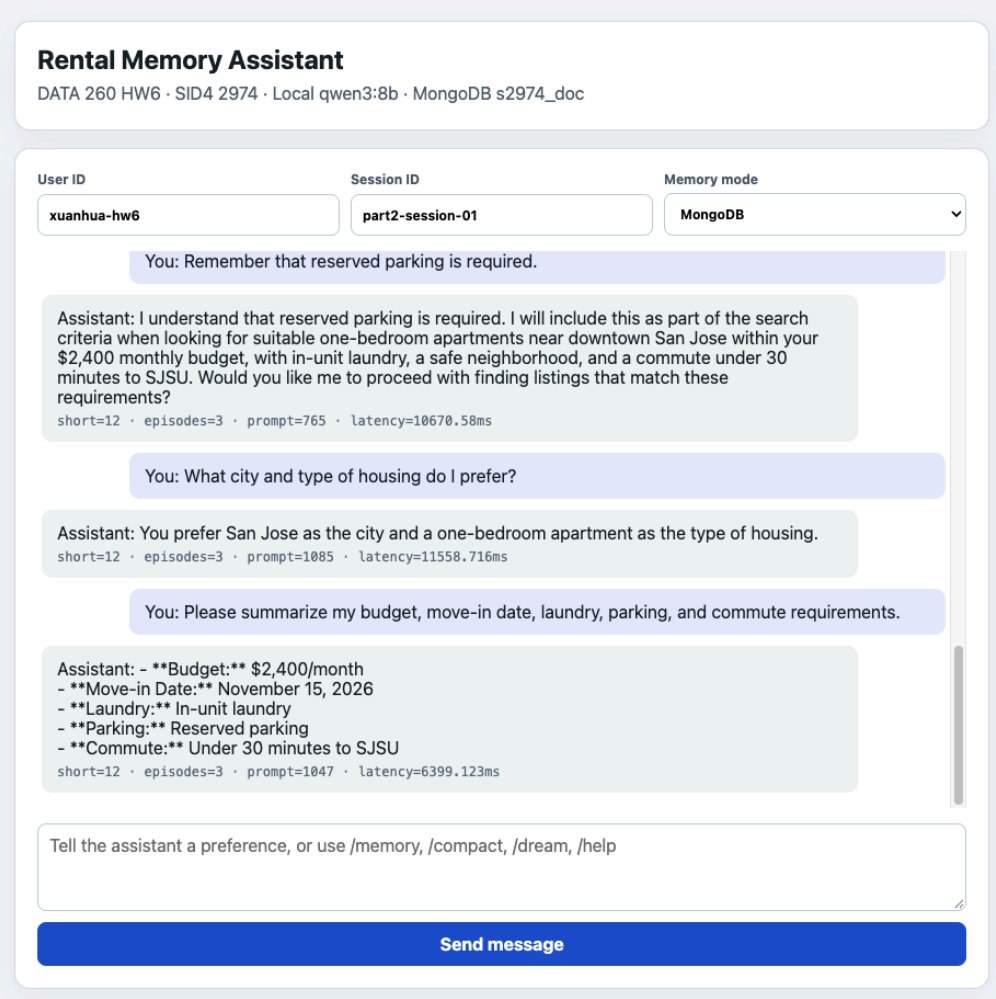


Figure 7. The chat interface recalls budget, location, parking, laundry, and commute requirements.



Figure 8. GET /api/memory returned HTTP 200 with messages, session and lifetime summaries, and episodes.

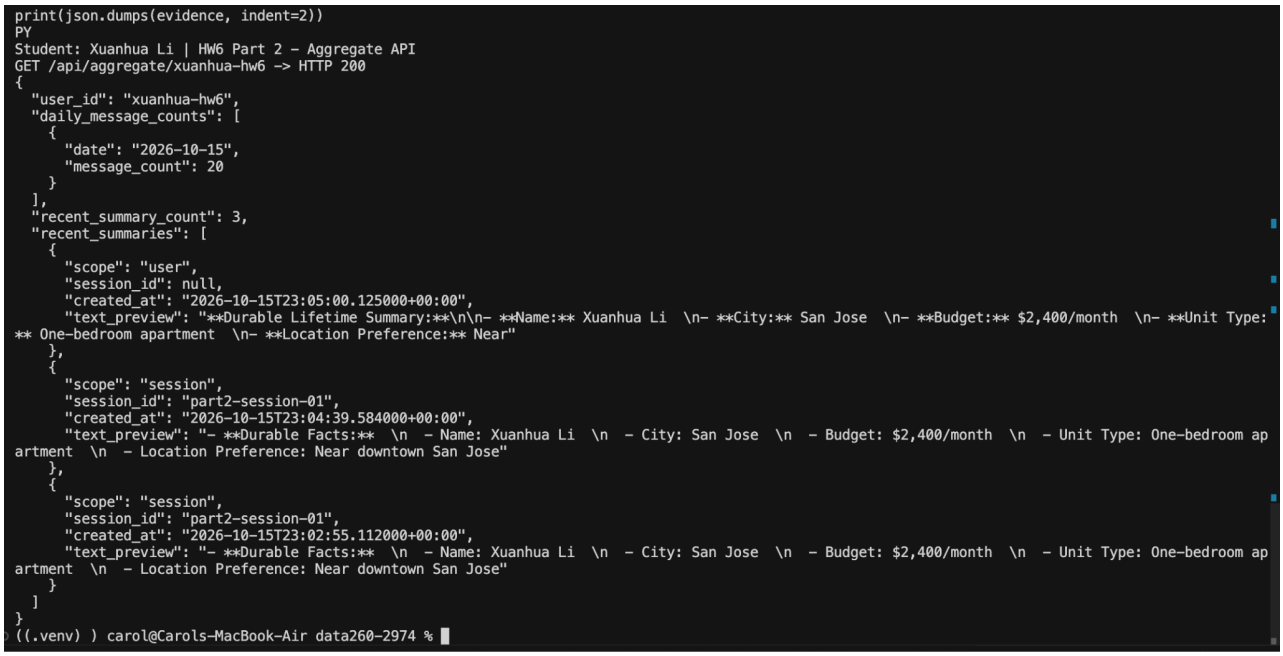


Figure 9. GET /api/aggregate returned 20 daily messages and three recent summaries.

Observed Results. The test session stored 20 messages: 10 user turns and 10 assistant replies. The memory endpoint returned the most recent 16 messages, the latest session summary, a lifetime summary, and 10 episodic facts. Each episodic embedding contained 384 dimensions.

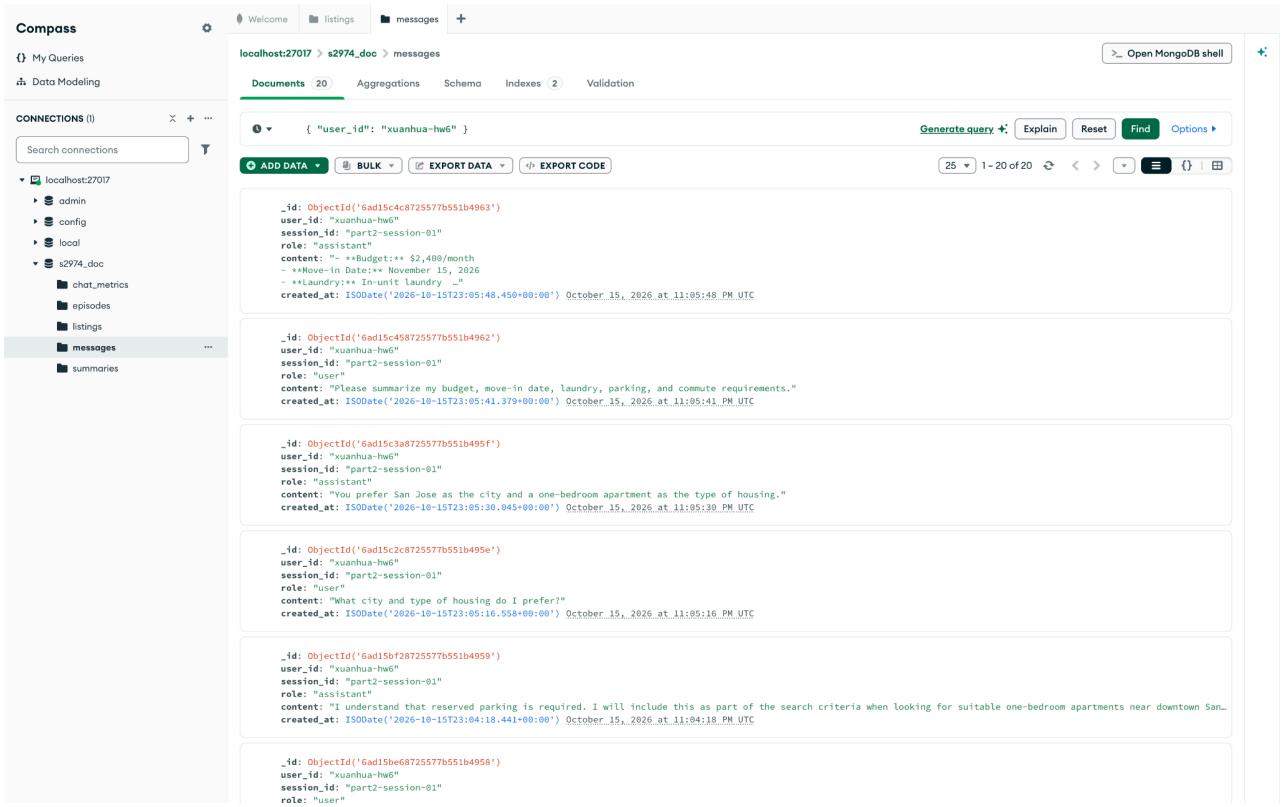


Figure 10. MongoDB Compass shows user and assistant records in the messages collection.

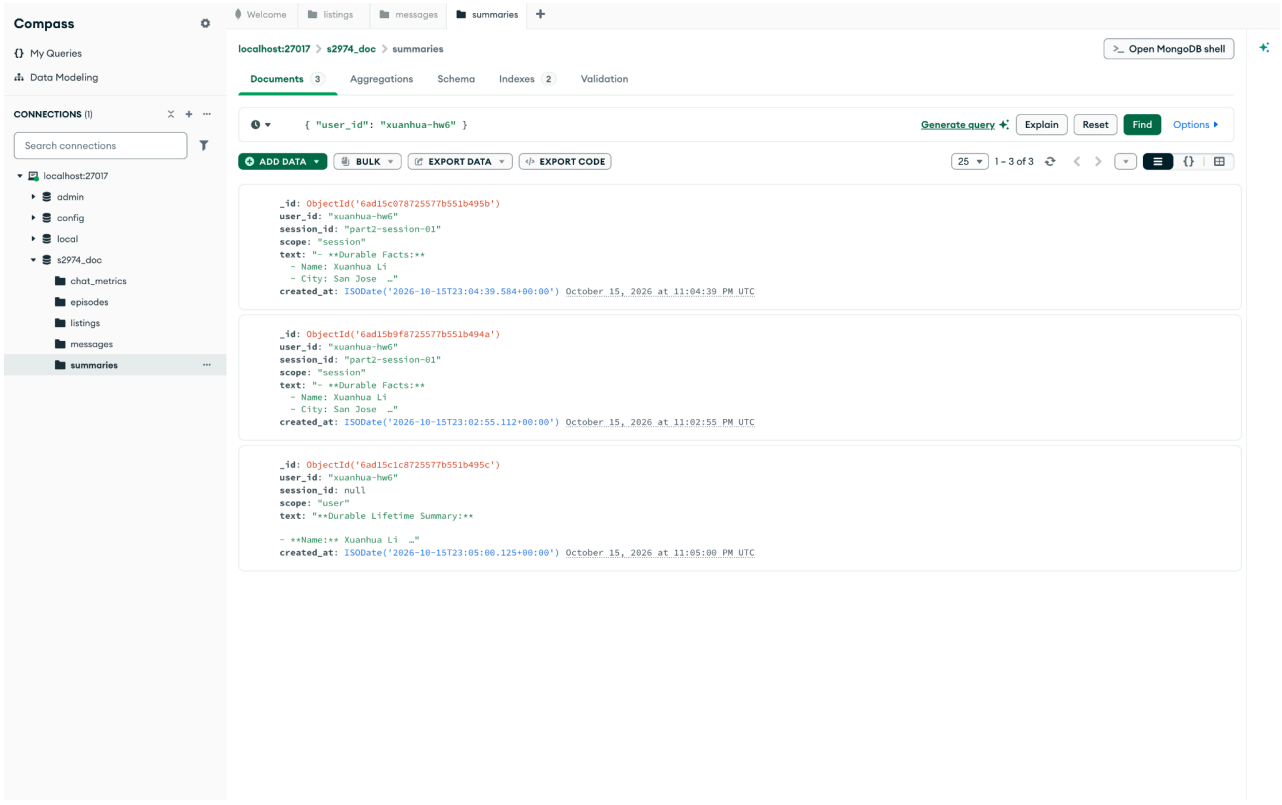


Figure 11. The summaries collection contains session- and user-scope documents.

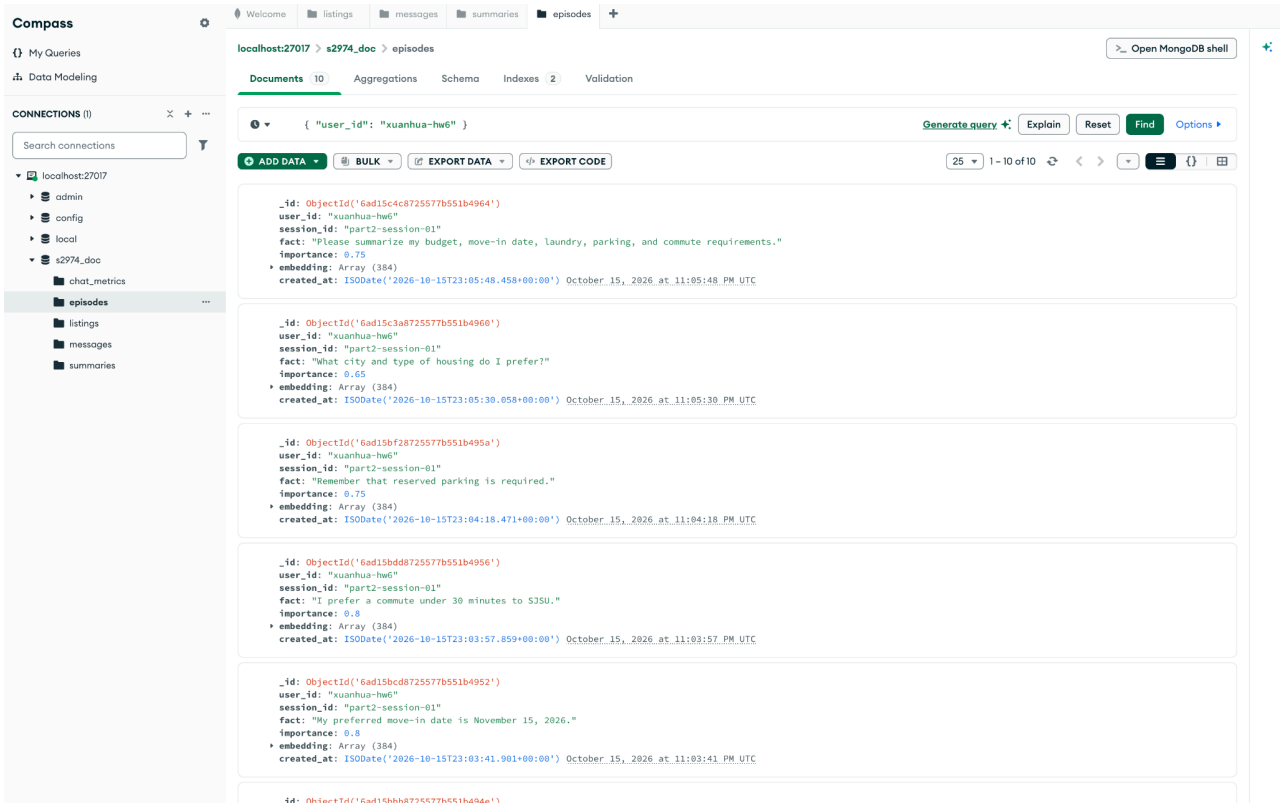


Figure 12. The episodes collection stores facts, importance scores, and 384-dimensional embeddings.

Part 3 What Memory Costs

Experiment Design. I ran the same 30-message rental conversation in fresh sessions with SHORT_TERM_N values of 4, 12, and 30. I then repeated the N=12 run with importance-based eviction. Every setting used the same five fixed probes and the same 1-5 scoring rubric.

Metrics and probes

Code snippet 8. Per-turn token, component, and latency logging

```
metric = {
    "prompt_tokens": result.input_tokens,
    "completion_tokens": result.output_tokens,
    "auxiliary_input_tokens": auxiliary_input_tokens,
    "auxiliary_output_tokens": auxiliary_output_tokens,
    "total_input_tokens": result.input_tokens + auxiliary_input_tokens,
    "total_output_tokens": result.output_tokens + auxiliary_output_tokens,
    "token_breakdown": token_breakdown,
    "latency_ms": round(latency_ms, 3),
}
repository.save_metric(metric)
self._write_metric(metric)
```

Code snippet 9. Example from memory_probes.yaml; the file contains five probes

```
scoring_rubric:
  5: Correct fact with no conflicting or outdated value.
  3: Correct fact appears together with a conflicting value.
  1: Incorrect fact, outdated fact, or no usable recall.
probes:
  - id: budget
    question: What is my current maximum monthly rent budget?
    expected_any: ["2200", "2,200"]
    forbidden_any: ["2400", "2,400"]
```

Experiment results

Setting	Prompt tokens	Latency ms	Quality	Estimated cost
SHORT_TERM_N=4	936.800	12,796.745	4.600	$727.92
SHORT_TERM_N=12	1,044.200	15,098.258	4.200	$688.01
SHORT_TERM_N=30	1,249.033	19,876.919	4.600	$751.19
Importance eviction N=12	911.567	14,989.820	4.600	$626.41

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 3 - Memory Cost Results"
cat reports/hw06/METRICS.md
Student: Xuanhua Li | HW6 Part 3 - Memory Cost Results
# HW6 Metrics

| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
|---|---|---|---|
| SHORT_TERM_N=4 | 936.800 | 12796.745 | 4.600 |
| SHORT_TERM_N=12 | 1044.200 | 15098.258 | 4.200 |
| SHORT_TERM_N=30 | 1249.033 | 19876.919 | 4.600 |
| Importance eviction (N=12) | 911.567 | 14989.820 | 4.600 |

## Eviction comparison

Prompt-token savings: 12.702%
Probe-quality change: 0.400

## Cost estimate

Best: Importance eviction (N=12) - $626.41
Worst: SHORT_TERM_N=30 - $751.19

Selection rule: Best is the lowest-cost setting within 0.5 points of the highest probe-quality score. Worst is the highest-cost measured setting.
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 13. Generated METRICS.md results for the four memory settings.

Memory-saving policy and cost

Code snippet 10. Importance-based short-term eviction policy

```
def importance_evict(messages: list[dict], keep: int = 6) -> list[dict]:
    scored = []
    for index, message in enumerate(messages):
        content = str(message.get("content", ""))
        recency = (index + 1) / len(messages)
        information = min(len(content) / 250, 1.0)
        fact_bonus = 0.5 if re.search(
            r"\d|prefer|need|must", content, re.IGNORECASE) else 0
        scored.append((recency + information + fact_bonus,
                       index, message))
    chosen = sorted(sorted(scored, reverse=True)[:keep],
                    key=lambda item: item[1])
    return [item[2] for item in chosen]
```

Cost formula. Estimated cost = 10,000 conversations x 30 turns x [(mean input tokens / 1,000,000) x $1 + (mean output tokens / 1,000,000) x $5]. The selection rule chooses the lowest-cost setting within 0.5 points of the highest probe-quality score.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 3 - Eviction and Cost Comparison"

python - <<'PY'
import json

with open("reports/hw06/raw/eviction_comparison.json") as f:
    eviction = json.load(f)

with open("reports/hw06/raw/memory_costs.json") as f:
    costs = json.load(f)

evidence = {
    "prompt_token_savings_percent": eviction["prompt_token_savings_percent"],
    "probe_quality_change": eviction["probe_quality_change"],
    "pricing": costs["pricing"],
    "selection_rule": costs["selection_rule"],
    "best": {
        "setting": costs["best"]["setting"],
        "probe_quality": costs["best"]["mean_probe_quality"],
        "estimated_cost_usd": costs["best"][
            "estimated_cost_10000_30_turn_conversations_usd"
        ],
    },
    "worst": {
        "setting": costs["worst"]["setting"],
        "probe_quality": costs["worst"]["mean_probe_quality"],
        "estimated_cost_usd": costs["worst"][
            "estimated_cost_10000_30_turn_conversations_usd"
        ],
    },
}

print(json.dumps(evidence, indent=2))
PY
Student: Xuanhua Li | HW6 Part 3 - Eviction and Cost Comparison
{
  "prompt_token_savings_percent": 12.702,
  "probe_quality_change": 0.4,
  "pricing": {
    "input_per_million_tokens_usd": 1,
    "output_per_million_tokens_usd": 5,
    "conversation_count": 10000,
    "turns_per_conversation": 30
  },
  "selection_rule": "Best is the lowest-cost setting within 0.5 points of the highest probe-quality score. Worst is the highest-cost measured setting.
",
  "best": {
    "setting": "Importance eviction (N=12)",
    "probe_quality": 4.6,
    "estimated_cost_usd": 626.41
  },
  "worst": {
    "setting": "SHORT_TERM_N=30",
    "probe_quality": 4.6,
    "estimated_cost_usd": 751.19
  }
}
```

Figure 14. Eviction savings, pricing assumptions, and best/worst setting selection.

Analysis. Increasing the short-term window generally increased prompt size and latency. N=30 was the worst setting because it had the highest prompt use, latency, and cost without improving quality beyond 4.6. Importance eviction was the best setting: it reduced prompt tokens by 12.702% relative to standard N=12 and increased probe quality from 4.2 to 4.6. N=4 had fewer prompt tokens than N=12 but a higher total cost because it produced more output tokens, which were priced five times higher than input tokens.

Part 4 File Based Memory and Consolidation

The second memory system loads durable project rules from AGENT.md and learned facts from MEMORY.md. Consolidation is required when MEMORY.md contains more than 40 non-empty, non-comment lines or when its last-consolidated timestamp is more than 24 hours old.

File loading and commands

Code snippet 11. Prompt loading and consolidation checks

```
def load_prompt_context(self) -> str:
    return (
        "Durable project instructions:\n" + self.load_agent() +
        "\n\nDurable learned facts:\n" + self.load_memory()
    )

def consolidation_status(self) -> dict[str, Any]:
    line_count = len(self.non_empty_non_comment_lines())
    age_exceeded = last_consolidated is None or (
        now - last_consolidated > timedelta(hours=24)
    )
    reasons = []
    if line_count > 40: reasons.append("more_than_40_memory_lines")
    if age_exceeded: reasons.append("older_than_24_hours")
    return {"required": bool(reasons), "line_count": line_count,
            "reasons": reasons}
```

Command	Behavior
/memory	Display durable memory and current consolidation status
/compact	Deduplicate, resolve contradictions, and report before/after counts
/dream	Extract useful durable facts from recent chat and append them
/help	Display all supported commands

Code snippet 12. Representative command handling

```
if normalized == "/memory":
    status = self.consolidation_status()
    return (
        f"{self.load_memory()}\n\nMemory status: "
        f"{status['line_count']} lines; "
        f"consolidation_required={status['required']}"
    )
if normalized == "/compact":
    result = self.consolidate()
    return (
        f"MEMORY.md consolidated: {result['before']} -> "
        f"{result['after']} non-empty, non-comment lines; "
        f"contradictions={len(result['contradictions'])}."
    )
if normalized == "/dream":
    result = self.dream(recent_messages)
    return f"Dream completed: added {result['added_count']} durable fact(s)."
```

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 4 - File Memory Consolidation"

python - <<'PY'
import json

with open("/tmp/hw6_compact.json") as f:
    data = json.load(f)

print("Command: POST /api/chat with /compact")
print("API reply:", data["reply"])
print("Current line count:", data["used_memory"]["file_memory_status"]["line_count"])
print(
    "Consolidation still required:",
    data["used_memory"]["file_memory_status"]["required"],
)
print("Prompt tokens:", data["metrics"]["prompt_tokens"])
PY
Student: Xuanhua Li | HW6 Part 4 - File Memory Consolidation
Command: POST /api/chat with /compact
API reply: MEMORY.md consolidated: 55 -> 29 non-empty, non-comment lines; contradictions=4.
Current line count: 29
Consolidation still required: False
Prompt tokens: 0
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 15. /compact reduced MEMORY.md from 55 to 29 lines and reported four contradictions.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 4 - Memory Commands"
echo "/memory -> HTTP $MEMORY_CMD_STATUS"
echo "/help   -> HTTP $HELP_CMD_STATUS"

python - <<'PY'
import json

with open("/tmp/hw6_memory_command.json") as f:
    memory = json.load(f)

with open("/tmp/hw6_help_command.json") as f:
    help_result = json.load(f)

status = memory["reply"].rsplit("Memory status:", 1)[-1].strip()
resolutions = [
    line
    for line in memory["reply"].splitlines()
    if line.startswith("- resolution_")
]

print("\n/memory status:")
print(status)

print("\nExplicit contradiction resolutions:")
for line in resolutions:
    print(line)

print("\n/help response:")
print(help_result["reply"])
PY
Student: Xuanhua Li | HW6 Part 4 - Memory Commands
/memory -> HTTP 200
/help   -> HTTP 200

/memory status:
29 lines; consolidation_required=False

Explicit contradiction resolutions:
- resolution_preferred_city: kept San Jose; superseded Santa Clara; rule=latest explicit value wins
- resolution_monthly_budget: kept 2200 USD; superseded 2400 USD; rule=latest explicit value wins
- resolution_move_in_date: kept 2026-11-15; superseded 2026-11-01; rule=latest explicit value wins
- resolution_pet: kept one indoor cat; superseded no pet; rule=latest explicit value wins

/help response:
/memory - display durable file memory
/compact - deduplicate and consolidate MEMORY.md
/dream - extract useful durable facts from recent chat
/help - display these commands
```

Figure 16. /memory displayed the final status and explicit resolutions; /help listed all commands.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 4 - Dream Command"
echo "/dream -> HTTP $DREAM_STATUS"

python - <<'PY'
import json

with open("/tmp/hw6_dream_command.json") as f:
    data = json.load(f)

print("API reply:", data["reply"])
print(
    "Memory line count after /dream:",
    data["used_memory"]["file_memory_status"]["line_count"],
)
print(
    "Consolidation required:",
    data["used_memory"]["file_memory_status"]["required"],
)
PY

echo
echo "Facts appended by /dream:"
sed -n '/## Facts added by \/dream/,$p' MEMORY.md
Student: Xuanhua Li | HW6 Part 4 - Dream Command
/dream -> HTTP 200
API reply: Dream completed: added 3 durable fact(s).
Memory line count after /dream: 32
Consolidation required: False

Facts appended by /dream:
## Facts added by /dream

- learned fact: I prefer a one-bedroom apartment near downtown San Jose.
- learned fact: I need in-unit laundry and a safe neighborhood.
- learned fact: I prefer a commute under 30 minutes to SJSU.
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 17. /dream extracted and appended three durable rental facts.

Consolidation results

Fact	Superseded value	Final value	Resolution rule
Preferred city	Santa Clara	San Jose	Latest explicit value wins
Monthly budget	$2,400	$2,200	Latest explicit value wins
Move-in date	Nov. 1, 2026	Nov. 15, 2026	Latest explicit value wins
Pet	No pet	One indoor cat	Latest explicit value wins

Consolidation reduced the fixture from 55 to 29 non-empty, non-comment lines. Duplicate facts were merged, and all four contradictions were preserved as explicit resolution records. No contradiction was identified as incorrectly resolved in this test.

AGENT instructions and verification

Category	Implemented instruction
Project description	Maintain a DATA 260 rental-housing-listing application
Technology stack	Use Python, FastAPI, MySQL, MongoDB, React, and local Ollama
Coding and testing	Use type hints and concise docstrings; add deterministic offline tests and an end-to-end smoke test
Prohibited actions	Do not send listing or memory data to a paid hosted model

Code snippet 13. Function implemented and tested against AGENT.md

```
def normalize_listing_title(value: str) -> str:
    """Return a clean rental-listing title of at most 100 characters."""
    if not isinstance(value, str):
        raise TypeError("listing title must be a string")
    cleaned = re.sub(r"\s+", " ", value).strip()
    if not cleaned:
        raise ValueError("listing title cannot be blank")
    if len(cleaned) > 100:
        raise ValueError("listing title cannot exceed 100 characters")
    return cleaned
```

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % echo "Student: Xuanhua Li | HW6 Part 4 — AGENT Compliance and Tests"

cat reports/hw06/AGENT_COMPLIANCE.md

echo
echo "Running deterministic HW6 offline tests..."
python code/test_hw6_offline.py
Student: Xuanhua Li | HW6 Part 4 — AGENT Compliance and Tests
# AGENT.md Function Verification

Function checked: `normalize_listing_title` in `code/hw6_agent_compliant.py`.

- Project description: the function validates the primary title field of the rental-listing domain.
- Technology stack: it is implemented in Python and can be called by the existing FastAPI application.
- Coding conventions: it has type hints, a concise docstring, a single responsibility, and clear validation errors.
- Testing approach: `code/test_hw6_offline.py` checks normal, blank, non-string, and over-100-character inputs.
- Prohibited actions: it performs no network requests, reads no secrets, and sends no data to a hosted model.

Running deterministic HW6 offline tests...
test_agent_compliant_function (__main__.HW6OfflineTests.test_agent_compliant_function) ... ok
test_cosine_and_token_allocation (__main__.HW6OfflineTests.test_cosine_and_token_allocation) ... ok
test_document_listing_validation (__main__.HW6OfflineTests.test_document_listing_validation) ... ok
test_episode_extraction_and_eviction (__main__.HW6OfflineTests.test_episode_extraction_and_eviction) ... ok
test_file_consolidation_and_commands (__main__.HW6OfflineTests.test_file_consolidation_and_commands) ... ok
test_memory_service_without_external_services (__main__.HW6OfflineTests.test_memory_service_without_external_services) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.008s

OK
HW6 OFFLINE TESTS: 6/6 PASS
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

Figure 18. AGENT.md compliance review and deterministic offline tests completed with 6/6 PASS.

MongoDB and file memory comparison

Criterion	MongoDB-backed memory	AGENT.md and MEMORY.md
Inspectability	Requires Compass, shell, or an API query	Immediately readable and editable Markdown
Persistence	Persists in the MongoDB Docker volume	Persists on disk and is versioned with Git
Scalability	Indexed collections support many users and sessions	A growing file becomes costly to load into each prompt
Must-not-forget facts	Reliable storage, but similarity retrieval may miss a fact	Strong for a small critical set always included in the prompt

Conclusion. A hybrid design is the best fit: project rules and a small set of critical durable facts remain in the Markdown files, while messages, summaries, embeddings, and per-user episodic memories are stored in MongoDB.

AI Use

1 What I used an AI assistant for and what I did myself

I used Codex to assist with implementing and reviewing parts of the MongoDB and memory-system code and troubleshooting integration issues with the existing project. I ran the services locally, inspected the API and MongoDB results, executed the experiments, collected the screenshots, reviewed the reported metrics, and verified the final submission.

2 One output I independently verified

I independently verified whether the provided long-term-memory demo could be copied directly. The demo used Flask and a hosted OpenAI model, while HW6 required the existing FastAPI application and local Ollama adapter. I also found that the initial Uvicorn launch configuration could not import the root-level src package.

3 How I detected and verified the issue

I compared the demo architecture with the HW6 requirements and my existing project structure. I then ran the application locally and observed ModuleNotFoundError: No module named 'src'. After adjusting the launch environment, I confirmed the solution through the /health endpoint, API tests, MongoDB Compass, and the final 9/9 smoke verification.

4 What I changed and why it works

I retained FastAPI and the existing src/model_client.py local Ollama adapter, then integrated MongoDB-backed memory around that structure. I launched the application with PYTHONPATH set to both the repository root and the code directory. This preserved the existing architecture while allowing the HW6 routers and model adapter to load correctly.

Final Verification

The final smoke script checked repository files, offline logic, MongoDB collections, API health, CRUD behavior, validation, both memory systems, and command support. All nine checks passed. The machine-readable result is stored in reports/hw06/verification.json.

